

Video Script & Protocol

Episode 1: Image Recognition with Local Vision LLM (qwen2.5v1)

Channel: Data, Paper & Code

September 17, 2026

Contents

1	Structure & Thematic Outline	2
2	YouTube Speaker Script (Voiceover Text)	2
3	Theoretical Background	3
3.1	Why Automated Image Recognition?	3
3.2	VLM Architecture: How Models Understand Images	3
3.3	Why Base64 Conversion?	4
4	System Setup & Ollama Guide	5
5	Full Python Script	5
6	Results & Model Output	8
7	Outlook & Application Domains	8

1 Structure & Thematic Outline

No.	Topic Block	Core Content & Objective
01	Welcome	Unconventional hook & channel intro.
02	Automated Image Recognition	Why visual data analysis is essential in data pipelines.
03	What is an LLM	Working principles and limitations of classic text models.
04	Multimodal LLMs	How language models learn to process images as input.
05	Local LLMs	Data privacy, independence, and hardware requirements.
06	Ollama Setup	Installation and background operation of Ollama.
07	Model Selection	Deployment of the multimodal model <code>qwen2.5v1</code> .
08	Python Integration	REST API calls (<code>/api/generate</code>) via <code>requests</code> .
09	Base64 Encoding	Why and how images are converted into ASCII text.
10	Architecture Deep Dive	VLM Training: Vision Encoders (ViT/CLIP), Projection Layers & Text LLMs.
11	Setup & Model Download	Installing Ollama & downloading <code>qwen2.5v1</code> .
12	Practical Live Code	Automated generation and analysis of a financial chart.

Table 1: Structure and schedule of the first video episode

2 YouTube Speaker Script (Voiceover Text)

Part 1: The Unconventional Hook & Intro

[Camera: Close-up | Dynamic & Thoughtful]

"Imagine handing an AI a 500-page book — and within two seconds, it delivers a flawless summary of the entire work. But as soon as you show it a simple image of a fever curve or a stock chart, it suddenly says: *'Sorry, I'm just a text model, I don't have eyes.'*

Crazy, right? We feed AIs with terabytes of text, but when faced with simple images, they've been completely blind for years.

Time to change that. Welcome to **Data, Paper & Code**! Today, we break this paradigm. We are building our own, completely locally running image recognition system directly on your machine. No cloud subscriptions, no data leaks overseas, no API costs. Just Python, Ollama, and the brand-new `qwen2.5v1`. Buckle up, let's dive right in!"

Part 2: Theory — How an AI "Eats" Images

[Screen: Architecture Diagram / ViT Patches Overlay]

"Before we run the code, we need to understand a fundamental hurdle: An LLM only understands tokens — vector numbers representing words. An image, however, is a grid of pixels. How do we get a text model to actually 'see' pixels?

We use three core components: 1. The **Vision Encoder** — specifically a Vision Transformer (ViT). It slices our image into a checkerboard pattern of small tiles called patches. Each patch is treated like an individual word token. 2. The **Projection Layer** — this is

the mathematical bridge. It translates the image vectors directly into the language of the language model. 3. The **Autoregressive LLM** — which ultimately combines both worlds and explains word for word what is visible in the image."

Part 3: Installation & Model Setup (Ollama)

[Screen: Browser ollama.com / Terminal]

"To run all of this locally on our own PC, we need two things: The engine and the model.

1. ****Install Ollama:**** Go to ollama.com, download the installer for Windows, Mac, or Linux, and run it. Once Ollama is running, you'll see the small llama icon in your system tray on the bottom right. That means: Your local API server is up and running on port 11434.
2. ****Pull the Vision Model:**** Now open a terminal or PowerShell and type the following command:

```
ollama run qwen2.5v1
```

Ollama will automatically download the multimodal **qwen2.5v1** model and set it up. Once finished, the interface is ready, and we can jump straight into Python!"

Part 4: The Base64 Trick & Live Code

[Screen: VS Code / IDE Open]

"Now comes the practical trick. When making REST API calls, we don't want to handle complex file uploads. We take the image file, run it through **Base64 encoding**, and turn the binary image into a pure text string of ASCII characters.

We embed this string directly into the JSON payload of our request to Ollama. Let me show you the Python code: We fetch market data via **yfinance**, generate a chart using **matplotlib**, encode it in Base64, and send it to <http://localhost:11434/api/generate>. Let's run the script... and BOOM! Look at the terminal output: The model not only identified the upward trend, but it also pinpointed the exact breakout points at marks 50 and 70. Fully automated, in seconds, running on your local CPU!"

3 Theoretical Background

3.1 Why Automated Image Recognition?

In scientific papers, financial reports, and experimental measurements, data often exists in visual formats (charts, PDFs, diagrams). Automated analysis enables seamless information extraction without requiring manual interpretation.

3.2 VLM Architecture: How Models Understand Images

Vision LLMs (VLMs) like **qwen2.5v1** combine visual signal processing with natural language processing through three core stages:

1. Vision Encoder (Extracting Visual Features):

- **ViT (Vision Transformer):** Splits the input image into a grid of small patches (e.g., 16×16 pixels). Each patch is treated like a word token, allowing self-attention mechanisms to learn spatial relationships within the image.

- **CLIP (Contrastive Language-Image Pre-training):** A model trained to align image and text embeddings within the same vector space, enabling semantic understanding of image content in relation to language.

2. Projection Layer / Cross-Attention (The Interface):

Since the vision encoder and language model operate in different vector spaces, this component serves as the mathematical bridge:

- **Projection Layer (MLP):** Acts as a "translator" that projects the vector dimensions of the vision encoder (e.g., 768) to match the expectations of the text LLM (e.g., 4096), processing image tokens as visual words.
- **Cross-Attention:** Dynamically connects text and image tokens. Text tokens act as queries, while image tokens provide keys and values. This directs model attention specifically to relevant image areas (e.g., trend lines in a chart).

3. Autoregressive LLM (Response Generation):

The language model processes the translated image tokens alongside the text prompt within the same context window, generating the final analysis token by token.

3.3 Why Base64 Conversion?

HTTP REST APIs send JSON payloads, which consist of plain text. Since raw binary image data contains special characters that would break JSON formatting, the image is encoded into a **Base64 ASCII string**.

Detailed technical background:

- **JSON Compatibility & Syntax Protection:** Binary files (e.g., PNG or JPEG) contain arbitrary byte values (0x00 to 0xFF), including null bytes, control characters, quotation marks ("), and backslashes (\). These would corrupt the JSON format or cause parsing errors.
- **Safe 7-Bit ASCII Transmission:** Base64 converts every 3 bytes of binary data (24 bits) into 4 printable characters. The character set is restricted to 64 neutral characters (A-Z, a-z, 0-9, +, /, and = for padding), guaranteeing corruption-free HTTP network transfer.
- **Single-Payload Structure:** The image can be passed directly as a simple string value inside the JSON body ({"images": ["iVBORw0KG..."]}), eliminating the need for complex data types or multipart/form-data headers.
- **Trade-off (Overhead):** The 3-to-4-byte encoding scheme increases the transferred data size by approximately 33 %. For API prompts and individual charts, this overhead is negligible.

```

1 import base64
2
3 def encode_image_to_base64(image_path: str) -> str:
4     """Encodes a local image into a Base64 string."""
5     with open(image_path, "rb") as image_file:
6         return base64.b64encode(image_file.read()).decode('utf-8')

```

Listing 1: Standalone Function: Convert Image to Base64

4 System Setup & Ollama Guide

Guide: Starting & Setting Up Ollama

1. **Download & Installation:** Download and install Ollama from ollama.com.
2. **Check Status:** Verify the Ollama icon in the system tray (taskbar).
3. **Manual Start:** If inactive, open a terminal and run `ollama serve`.
4. **Load Model:** Run the following command once in the console:

```
ollama run qwen2.5vl
```
5. **API Interface:** The Python script targets `http://localhost:11434/api/generate`.

5 Full Python Script

The following script generates a chart (e.g., financial market data), encodes it as Base64, and submits it to the local `qwen2.5vl` model.

```
1 import base64
2 import os
3 import matplotlib.pyplot as plt
4 import requests
5 import yfinance as yf
6
7
8 # =====
9 # 1. CREATE IMAGE (e.g. Stock Chart)
10 # =====
11 def generate_chart_image(ticker: str = "VUSD.L", filename: str = "chart.
    png") -> str:
12     """Downloads market data, generates a chart image, and saves it."""
13     data = yf.download(ticker, period="3mo")
14
15     plt.figure(figsize=(10, 5), dpi=150)
16     plt.plot(data.index, data['Close'], label=f"{ticker} Price History",
17             color='#1f77b4', lw=2)
18     plt.title(f"Current Trend for {ticker}", fontsize=14, fontweight='bold')
19     plt.xlabel("Date")
20     plt.ylabel("Price")
21     plt.grid(True, linestyle='--', alpha=0.6)
22     plt.tight_layout()
23
24     plt.savefig(filename)
25     plt.close()
26     print(f"[INFO] Chart saved as image: {filename}")
27     return filename
28
29 # =====
30 # 2. CONVERT IMAGE TO BASE64
31 # =====
```

```

32 def encode_image_to_base64(image_path: str) -> str:
33     """Encodes a local image into a Base64 string."""
34     with open(image_path, "rb") as image_file:
35         return base64.b64encode(image_file.read()).decode('utf-8')
36
37
38 # =====
39 # 3. SEND IMAGE TO VISION LLM IN OLLAMA
40 # =====
41 def analyze_chart_with_ollama(
42     image_path: str,
43     ollama_url: str = "http://localhost:11434/api/generate",
44     model_name: str = "qwen2.5vl"
45 ) -> str:
46     """Sends the image to the Vision model hosted in Ollama."""
47
48     # Convert image to Base64
49     base64_image = encode_image_to_base64(image_path)
50
51     prompt_text = (
52         "You are a technical chart analyst. Analyze the attached chart\n"
53         "image:\n"
54         "1. What trend (Upward, Downward, Sideways) is visible?\n"
55         "2. Identify notable breakouts or consolidations along the\n"
56         "trajectory.\n"
57         "3. Provide a concise summary of the visual situation in 3\n"
58         "sentences."
59     )
60
61     payload = {
62         "model": model_name,
63         "prompt": prompt_text,
64         "images": [base64_image],
65         "stream": False,
66         "options": {
67             "temperature": 0.2
68         }
69     }
70
71     try:
72         print(f"\n[INFO] Sending image to Ollama ({model_name})...")
73         response = requests.post(ollama_url, json=payload)
74         response.raise_for_status()
75
76         result_json = response.json()
77         return result_json.get("response", "No response received from\n"
78                                "model.")
79
80     except requests.exceptions.RequestException as e:
81         return f"[ERROR]: Request to Ollama failed: {e}"
82
83
84 def print_ollama_instructions():
85     instructions = """
86     =====
87     INSTRUCTIONS: STARTING & SETTING UP OLLAMA
88     =====
89     1. CHECK OLLAMA IN SYSTEM TRAY:
90     - Check bottom right in the Windows system tray (next to the clock).

```

```

87     - Is the Ollama icon visible? -> Ollama is already running!
88
89 2. START OLLAMA MANUALLY (if inactive):
90     - Option A: Open Windows Start Menu and click "Ollama".
91     - Option B: Open any terminal / PowerShell and type:
92         ollama serve
93
94 3. DOWNLOAD MODEL ONCE (if not done yet):
95     - Open PowerShell and execute the following command:
96         ollama run qwen2.5vl
97
98 4. SCRIPT PREREQUISITE:
99     - The Python script sends requests to: http://localhost:11434
100    - As long as Ollama runs in the background, image analysis
101      will process successfully.
102    =====
103    """
104        print(instructions)
105
106
107    # =====
108    # MAIN EXECUTION EXAMPLE
109    # =====
110    if __name__ == "__main__":
111        # Print instructions to console upfront
112        print_ollama_instructions()
113
114        # 1. Define image path or generate chart
115        chart_file = "Kurve.jpg"
116
117        if not os.path.exists(chart_file):
118            chart_file = generate_chart_image(ticker="VUSD.L", filename="chart
119            .png")
120
121        # 2. Evaluate graphic via Ollama
122        analysis_result = analyze_chart_with_ollama(chart_file)
123
124        print("\n" + "=" * 50)
125        print("    VISUAL RECOGNITION FROM LLM (OLLAMA):")
126        print("=" * 50)
127        print(analysis_result)

```

Listing 2: Complete Source Code for Automated Chart Analysis

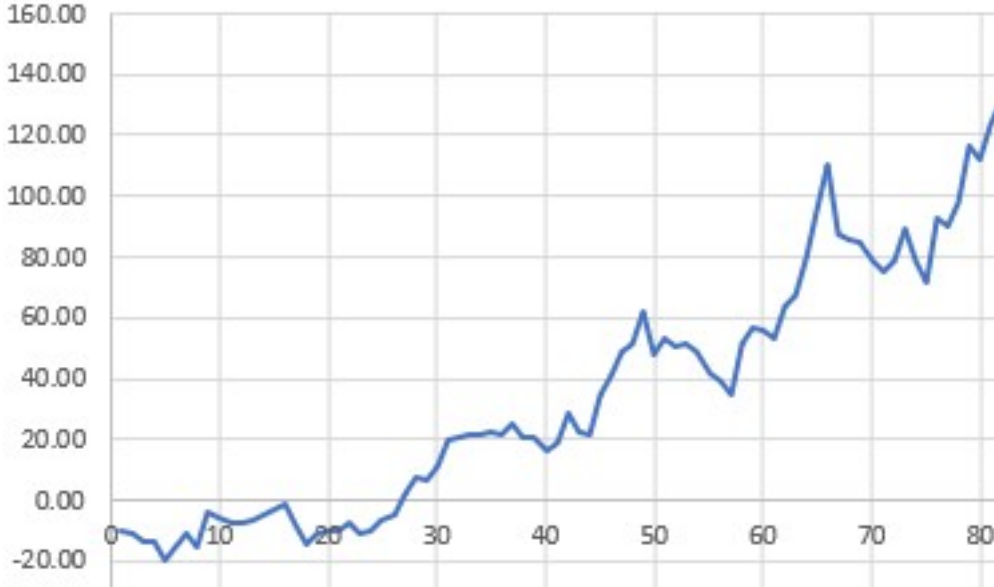


Figure 1: Chart selected for analysis.

6 Results & Model Output

Terminal output after executing the script

Command: python Image01.py

[INFO] Sending image to Ollama (qwen2.5v1)...

VISUAL RECOGNITION FROM LLM (OLLAMA):

1. The trend of the attached chart is **Upward**. The curve rises continuously, indicating a positive development.
2. **Notable Breakouts:** A significant increase occurs between the 50 and 60 marks, followed by another strong upward movement after the 70 mark. These points could serve as key support or resistance levels.
3. **Summary:** The chart demonstrates a clear upward trajectory, featuring two prominent breakouts at marks 50 and 70. Overall, the curve exhibits steady growth, pointing toward a positive overall trend.

7 Outlook & Application Domains

The captured measurement curve and its mathematical analysis form an essential foundation for practical applications in research and industry:

- **Model Validation & Simulation:** Aligning theoretical simulation models with real experimental data to identify system limits and parameter deviations.
- **Quality Assurance & Process Monitoring:** Establishing tolerance bands and reference curves to detect manufacturing defects or material flaws early.
- **Regression Diagnostics & Error Analysis:** Evaluating residuals (e.g., via *Normalized Residual Plots*) to identify systematic measurement errors and sensor drifts.

- **Integration into AI & RAG Pipelines:** Extracting curve trajectories and parameters for automated documentation, pattern recognition, and hypothesis generation within analytical systems.